

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-aecc45ad6cbb9cbab.jsonl`  
- SHA-256: `c834cd8fa36c74ab73e974e19897aa0bc3363d5eef34a6720926b55b5b5f93cf`  
- Source modified: `2026-07-03T17:36:21+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-03T17:33:09.393Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer, checked out at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer on branch codex/a2-golden-manifest-portability (HEAD 5a91701, one commit over origin/master). Get the diff: `git diff origin/master..HEAD`. The PR adds backend/eval/golden_manifest.py (a portable golden-corpus manifest resolver: rebases stale absolute paths against the local workspace by searching candidate dirs, fails loudly on missing trajectory/labels files), wires 8 consumers (calibrate_local, distill_local, fusion_golden, rally_seg_eval, rally_seq_proto, serve_contrast, served_gate_a, training/gen0/harness, plus tests/test_served_gate_a_regression helper) through load_resolved_manifest, and adds tests + docs.

ANGLE A ? line-by-line scan. Read every hunk, then Read the enclosing function of each hunk. For every line ask what input/state/platform makes it wrong. Specifically check:

- backend/eval/golden_manifest.py end-to-end: `_candidate_paths/_candidate_dirs/_expand_root` ordering and dedup (could a WRONG file shadow the right one ? e.g., a same-basename file in manifest_dir winning over the correct Trajectories dir?); `_basename` with `PureWindowsPath` on edge inputs (trailing slash, no separators, POSIX absolute); `os.path.expandvars/expanduser` surprises; `repo_root()` parents[2] arithmetic; `ManifestPathError` message truncation; `write_manifest` atomicity (tmp name collision, `os.replace` cross-device); CLI `main()` exit codes, `--min-count` interaction with issues, the ``rebased`` zip count (does `zip(original, resolved)` stay aligned when rows were skipped?); `JSONDecodeError` inheritance from `ValueError` in the except clause (dead or fine?).
- Each consumer file: was ``import json`` removed while `json` is still used elsewhere in that file (grep the file for `json`. after the change)? Did the call-site change alter semantics (e.g., `rally_seg_eval.load_golden` now strict on trajectory only ? what does the enclosing loop do with labels)?
- tests: do the new/modified tests actually assert the loud-failure path (exit code, `stderr`)?

Return up to 6 candidates as a JSON array of {file, line, summary, failure_scenario}. line = new-file line number. Concrete failure scenarios only; pass through every candidate with a nameable scenario. [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:33:11.475Z)

I'll analyze PR #452 by examining the diff and checking for potential issues across the golden manifest implementation and its consumers.

3. user (2026-07-03T17:33:12.429Z)

backend/eval/calibrate_local.py		5 +-
backend/eval/distill_local.py		5 +-
backend/eval/fusion_golden.py		4 +-
backend/eval/golden_manifest.py		378 ++++++
backend/eval/rally_seg_eval.py		4 +-
backend/eval/rally_seq_proto.py		5 +-
backend/eval/serve_contrast.py		4 +-

```

backend/eval/served_gate_a.py          | 5 +-
docs/ALPHA_LAUNCH_READINESS.md         | 11 +-
docs/data_pipeline/GOLDEN_VIDEOS.md    | 8 +-
tests/test_fusion_compare.py           | 4 +
tests/test_fusion_golden_telemetry.py  | 3 +
tests/test_golden_manifest.py          | 158 ++++++
tests/test_served_gate_a_regression.py | 21 +-
training/gen0/harness.py               | 4 +-
15 files changed, 586 insertions(+), 33 deletions(-)

```

4. assistant (2026-07-03T17:33:15.089Z)

Now let me get the full diff to analyze all changes:

5. user (2026-07-03T17:33:15.911Z)

<persisted-output>

Output too large (33.2KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\b410yz9ja.txt

Preview (first 2KB):

```

diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
 from __future__ import annotations

 import argparse
-import json
 from typing import Callable, Dict, List, Tuple

 from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.detectors import rally_gate
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
 from __future__ import annotations

 import argparse
-import json
 from typing import Dict, List, Optional, Tuple

 from backend.eval import windowing as _windowing
 from backend.eval.calibrate_wasb import load_golden_gts

```

```

from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.training import label_candidates
from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
    )
...
</persisted-output>

```

6. user (2026-07-03T17:33:18.204Z)

```

1      diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
2      index 687f790..4734f38 100644
3      --- a/backend/eval/calibrate_local.py
4      +++ b/backend/eval/calibrate_local.py
5      @@ -24,7 +24,6 @@ where videos.json =
[{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
6      from __future__ import annotations
7
8      import argparse
9      -import json
10     from typing import Callable, Dict, List, Tuple
11
12     from backend.eval.calibrate_wasb import load_golden_gts
13     @@ -34,6 +33,7 @@ from backend.eval.calibration import (
14         leave_one_video_out,
15         select_best,
16     )
17 +from backend.eval.golden_manifest import load_resolved_manifest
18     from backend.pipeline.detectors import rally_gate
19     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
20
21     @@ -164,8 +164,7 @@ def main() -> None:
22         ap.add_argument("--iou", type=float, default=0.5)
23         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
24         args = ap.parse_args()
25         - with open(args.manifest, encoding="utf-8") as f:
26         -     specs = json.load(f)
27     +     specs = load_resolved_manifest(args.manifest)
28         run(specs, iou=args.iou, metric=args.metric)
29
30
31     diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
32     index 52ac85a..a05b3ff 100644
33     --- a/backend/eval/distill_local.py
34     +++ b/backend/eval/distill_local.py
35     @@ -24,13 +24,13 @@ Run:
36         from __future__ import annotations
37
38         import argparse
39     -import json
40         from typing import Dict, List, Optional, Tuple
41
42         from backend.eval import windowing as _windowing
43         from backend.eval.calibrate_wasb import load_golden_gts
44         from backend.eval.classifier import LogisticRegression
45         from backend.eval.gemini_refine import merge_overlaps
46     +from backend.eval.golden_manifest import load_resolved_manifest
47         from backend.eval.metrics import score
48         from backend.eval.training import label_candidates
49         from backend.pipeline.detectors import rally_gate
50     @@ -218,8 +218,7 @@ def main() -> None:
51         )

```

```

52         ap.add_argument("--model-out", default="output/local_rally_model.json")
53         args = ap.parse_args()
54         with open(args.manifest, encoding="utf-8") as f:
55             specs = json.load(f)
56         specs = load_resolved_manifest(args.manifest)
57         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
58
59
60 diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
61 index dd3f6a1..aee6d7e 100644
62 --- a/backend/eval/fusion_golden.py
63 +++ b/backend/eval/fusion_golden.py
64 @@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional
65
66     from backend.eval import distill_local
67     from backend.eval.calibrate_wasb import load_golden_gts
68 +from backend.eval.golden_manifest import load_resolved_manifest
69     from backend.eval.training import label_candidates
70     from backend.pipeline.detectors import fusion_features as ff
71     from backend.pipeline.detectors import rally_gate
72 @@ -160,8 +161,7 @@ def run(
73     fresh: bool = False,
74     smallest_first: bool = False,
75 ) -> List[str]:
76     with open(manifest_path, encoding="utf-8") as f:
77         specs = json.load(f)
78     specs = load_resolved_manifest(manifest_path)
79     if smallest_first:
80         # Process small videos first for quick feedback (the slow full match lands
last);
81         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
82 diff --git a/backend/eval/golden_manifest.py b/backend/eval/golden_manifest.py
83 new file mode 100644
84 index 0000000..7a01677
85 --- /dev/null
86 +++ b/backend/eval/golden_manifest.py
87 @@ -0,0 +1,378 @@
88 +"""Portable golden-corpus manifest loader and smoke checker.
89 +
90 +The owner-side golden manifest is intentionally gitignored and has historically
91 +carried absolute paths from whichever box ingested a clip. This module keeps that
92 +artifact usable after the workspace moves by resolving stale paths against the
93 +current repo layout and failing loudly when required corpus files are absent.
94 +
95 +Default local layout supported:
96 + - ../Annotation Setup/Trajectories/<trajectory>.csv
97 + - ../Annotation Setup/Golden Labelled/<labels>.csv
98 + - output/<labels>.csv
99 +
100 +Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
101 +"""
102 +
103 +from __future__ import annotations
104 +
105 +import argparse
106 +import json
107 +import os
108 +import sys
109 +from dataclasses import dataclass
110 +from pathlib import Path, PureWindowsPath
111 +from typing import Any, Iterable, Sequence
112 +
113 +DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
114 +SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"

```

```

115 +DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
116 +
117 +_FIELD_DIR = {
118 +     "trajectory": "Trajectories",
119 +     "labels": "Golden Labelled",
120 +}
121 +
122 +
123 +@dataclass(frozen=True)
124 +class ManifestIssue:
125 +     """One manifest path problem found by the resolver."""
126 +
127 +     name: str
128 +     field: str
129 +     path: str
130 +     reason: str
131 +
132 +     def message(self) -> str:
133 +         shown = f" {self.path!r}" if self.path else ""
134 +         return f"{self.name}: {self.field}{shown} - {self.reason}"
135 +
136 +
137 +class ManifestPathError(RuntimeError):
138 +     """Raised when a strict manifest load finds missing required files."""
139 +
140 +     def __init__(self, manifest_path: str | os.PathLike[str], issues:
Sequence[ManifestIssue]):
141 +         self.manifest_path = os.fspath(manifest_path)
142 +         self.issues = list(issues)
143 +         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
144 +         more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12}
more"
145 +
146 +         super().__init__(
147 +             f"golden manifest path check failed for {self.manifest_path}: "
148 +             f"{len(self.issues)} issue(s)\n{detail}{more}"
149 +         )
150 +
151 +def repo_root() -> Path:
152 +     """Return the repository root for this module."""
153 +
154 +     return Path(__file__).resolve().parents[2]
155 +
156 +
157 +def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
158 +     """Load the flat golden manifest list and validate its outer shape."""
159 +
160 +     path = Path(manifest_path)
161 +     with path.open(encoding="utf-8") as f:
162 +         data = json.load(f)
163 +         if not isinstance(data, list):
164 +             raise ValueError(f"golden manifest must be a JSON list: {path}")
165 +         out: list[dict[str, Any]] = []
166 +         for i, row in enumerate(data):
167 +             if not isinstance(row, dict):
168 +                 raise ValueError(f"golden manifest row {i} is not an object: {path}")
169 +             out.append(dict(row))
170 +         return out
171 +
172 +
173 +def resolve_manifest_paths(
174 +     manifest_path: str | os.PathLike[str],
175 +     *,
176 +     repo_root_path: str | os.PathLike[str] | None = None,
177 +     extra_roots: Iterable[str | os.PathLike[str]] = (),

```

```

178 +     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
179 + ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
180 +     """Load and rebase required path fields in a golden manifest.
181 +
182 +     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
183 +     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
184 +     existing local path. Missing required fields are reported in ``issues``.
185 +     """
186 +
187 +     manifest = load_manifest(manifest_path)
188 +     return resolve_manifest_specs(
189 +         manifest,
190 +         manifest_path,
191 +         repo_root_path=repo_root_path,
192 +         extra_roots=extra_roots,
193 +         required_fields=required_fields,
194 +     )
195 +
196 +
197 +def load_resolved_manifest(
198 +    manifest_path: str | os.PathLike[str],
199 +    *,
200 +    repo_root_path: str | os.PathLike[str] | None = None,
201 +    extra_roots: Iterable[str | os.PathLike[str]] = (),
202 +    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
203 +    strict: bool = True,
204 + ) -> list[dict[str, Any]]:
205 +     """Load a manifest with stale paths rebased; optionally raise on missing files."""
206 +
207 +     specs, issues = resolve_manifest_paths(
208 +         manifest_path,
209 +         repo_root_path=repo_root_path,
210 +         extra_roots=extra_roots,
211 +         required_fields=required_fields,
212 +     )
213 +     if strict and issues:
214 +         raise ManifestPathError(manifest_path, issues)
215 +     return specs
216 +
217 +
218 +def resolve_manifest_specs(
219 +    specs: Sequence[dict[str, Any]],
220 +    manifest_path: str | os.PathLike[str],
221 +    *,
222 +    repo_root_path: str | os.PathLike[str] | None = None,
223 +    extra_roots: Iterable[str | os.PathLike[str]] = (),
224 +    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
225 + ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
226 +     """Resolve an already-loaded manifest list."""
227 +
228 +     manifest = Path(manifest_path)
229 +     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
230 +     roots = [Path(p) for p in _env_roots()]
231 +     roots.extend(Path(p) for p in extra_roots)
232 +     required = tuple(required_fields)
233 +
234 +     resolved: list[dict[str, Any]] = []
235 +     issues: list[ManifestIssue] = []
236 +     for idx, spec in enumerate(specs):
237 +         row = dict(spec)
238 +         name = str(row.get("name") or f"row[{idx}]")
239 +         for field in required:
240 +             raw = row.get(field)
241 +             if not raw:
242 +                 issues.append(ManifestIssue(name, field, "", "required field missing"))

```

```

243 +             continue
244 +             hit = resolve_existing_path(
245 +                 str(raw),
246 +                 field,
247 +                 manifest_path=manifest,
248 +                 repo_root_path=root,
249 +                 extra_roots=roots,
250 +             )
251 +             if hit is None:
252 +                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
253 +             else:
254 +                 row[field] = str(hit)
255 +                 resolved.append(row)
256 +             return resolved, issues
257 +
258 +
259 +def resolve_existing_path(
260 +    raw_path: str,
261 +    field: str,
262 +    *,
263 +    manifest_path: str | os.PathLike[str],
264 +    repo_root_path: str | os.PathLike[str],
265 +    extra_roots: Iterable[str | os.PathLike[str]] = (),
266 +) -> Path | None:
267 +    """Resolve one manifest path field to an existing file, if possible."""
268 +
269 +    for candidate in _candidate_paths(
270 +        raw_path,
271 +        field,
272 +        Path(manifest_path),
273 +        Path(repo_root_path),
274 +        [Path(p) for p in extra_roots],
275 +    ):
276 +        if candidate.is_file():
277 +            return candidate.resolve()
278 +    return None
279 +
280 +
281 +def _env_roots() -> list[Path]:
282 +    raw = os.environ.get(SEARCH_ROOTS_ENV, "")
283 +    if not raw:
284 +        return []
285 +    return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
286 +
287 +
288 +def _basename(raw_path: str) -> str:
289 +    # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
290 +    win_name = PureWindowsPath(raw_path).name
291 +    return win_name or Path(raw_path).name
292 +
293 +
294 +def _candidate_paths(
295 +    raw_path: str,
296 +    field: str,
297 +    manifest_path: Path,
298 +    root: Path,
299 +    extra_roots: Sequence[Path],
300 +) -> list[Path]:
301 +    basename = _basename(raw_path)
302 +    raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
303 +    manifest_dir = manifest_path.resolve().parent
304 +    candidates: list[Path] = [raw]
305 +
306 +    if not raw.is_absolute():
307 +        candidates.extend([manifest_dir / raw, root / raw])

```

```

308 +
309 +     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
310 +         candidates.append(directory / basename)
311 +     return _dedup_paths(candidates)
312 +
313 +
314 +def _candidate_dirs(
315 +    field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
316 +) -> list[Path]:
317 +    field_dir = _FIELD_DIR.get(field)
318 +    dirs: list[Path] = [manifest_dir]
319 +    if field_dir:
320 +        dirs.extend(
321 +            [
322 +                root.parent / "Annotation Setup" / field_dir,
323 +                root.parent / "Annotation Setup" / "Collect" / field_dir,
324 +                root / "output",
325 +            ]
326 +        )
327 +    dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
328 +    return _dedup_paths(dirs)
329 +
330 +
331 +def _expand_root(root: Path, field: str) -> list[Path]:
332 +    field_dir = _FIELD_DIR.get(field)
333 +    dirs = [root, root / "output"]
334 +    if field_dir:
335 +        dirs.extend(
336 +            [
337 +                root / field_dir,
338 +                root / "Annotation Setup" / field_dir,
339 +                root / "Annotation Setup" / "Collect" / field_dir,
340 +            ]
341 +        )
342 +    return dirs
343 +
344 +
345 +def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
346 +    out: list[Path] = []
347 +    seen: set[str] = set()
348 +    for path in paths:
349 +        try:
350 +            key = os.path.normcase(os.path.abspath(os.fspath(path)))
351 +        except OSError:
352 +            key = os.fspath(path)
353 +        if key in seen:
354 +            continue
355 +        seen.add(key)
356 +        out.append(path)
357 +    return out
358 +
359 +
360 +def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
361 +    """Write a resolved manifest JSON atomically enough for local tooling."""
362 +
363 +    out = Path(path)
364 +    out.parent.mkdir(parents=True, exist_ok=True)
365 +    tmp = out.with_name(out.name + ".tmp")
366 +    with tmp.open("w", encoding="utf-8") as f:
367 +        json.dump(list(specs), f, indent=2)
368 +        f.write("\n")
369 +    os.replace(tmp, out)
370 +
371 +

```



```

372 +def build_parser() -> argparse.ArgumentParser:
373 +     p = argparse.ArgumentParser(
374 +         description="Check/rebase the local golden corpus manifest paths."
375 +     )
376 +     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
377 +     p.add_argument(
378 +         "--repo-root",
379 +         default=None,
380 +         help="repo root used for default sibling output/Annotation Setup lookups",
381 +     )
382 +     p.add_argument(
383 +         "--root",
384 +         action="append",
385 +         default=[],
386 +         help=(
387 +             "additional search root; repeatable. Also accepts roots via "
388 +             f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
389 +         ),
390 +     )
391 +     p.add_argument(
392 +         "--require",
393 +         action="append",
394 +         choices=sorted(_FIELD_DIR),
395 +         default=None,
396 +         help="path field to require; repeatable. Defaults to trajectory and labels.",
397 +     )
398 +     p.add_argument(
399 +         "--min-count",
400 +         type=int,
401 +         default=None,
402 +         help="fail unless at least this many manifest rows are present",
403 +     )
404 +     p.add_argument(
405 +         "--write",
406 +         default=None,
407 +         help="optional path to write the resolved local manifest JSON",
408 +     )
409 +     return p
410 +
411 +
412 +def main(argv: Sequence[str] | None = None) -> int:
413 +     args = build_parser().parse_args(argv)
414 +     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
415 +     try:
416 +         original = load_manifest(args.manifest)
417 +         resolved, issues = resolve_manifest_specs(
418 +             original,
419 +             args.manifest,
420 +             repo_root_path=args.repo_root,
421 +             extra_roots=args.root,
422 +             required_fields=required,
423 +         )
424 +     except (OSError, ValueError, json.JSONDecodeError) as e:
425 +         print(f"ERROR: {e}", file=sys.stderr)
426 +         return 2
427 +
428 +     if args.min_count is not None and len(resolved) < args.min_count:
429 +         issues.append(
430 +             ManifestIssue(
431 +                 "manifest",
432 +                 "rows",
433 +                 str(len(resolved)),
434 +                 f"expected at least {args.min_count} row(s)",
435 +             )
436 +         )

```

```

437 +
438 +     if issues:
439 +         print(
440 +             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
441 +             file=sys.stderr,
442 +         )
443 +         for issue in issues:
444 +             print(f" - {issue.message()}", file=sys.stderr)
445 +         return 2
446 +
447 +     rebased = sum(
448 +         1
449 +         for before, after in zip(original, resolved)
450 +         for field in required
451 +         if str(before.get(field, "")) != str(after.get(field, ""))
452 +     )
453 +     print(
454 +         f"OK: {args.manifest} has {len(resolved)} video(s); "
455 +         f"{len(resolved) * len(required)} required path(s) present; "
456 +         f"{rebased} path(s) rebased."
457 +     )
458 +     if args.write:
459 +         write_manifest(args.write, resolved)
460 +         print(f"wrote resolved manifest: {args.write}")
461 +     return 0
462 +
463 +
464 +if __name__ == "__main__":
465 +    raise SystemExit(main())
466 diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
467 index 0acca2d..3846ce0 100644
468 --- a/backend/eval/rally_seg_eval.py
469 +++ b/backend/eval/rally_seg_eval.py
470 @@ -27,6 +27,7 @@ import os
471     from typing import Any, Dict, List, Optional, Tuple
472
473     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
474 +from backend.eval.golden_manifest import load_resolved_manifest
475     from backend.eval.metrics import Interval
476     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
477
478 @@ -185,8 +186,7 @@ def load_golden(
479     ) -> List[Dict[str, Any]]:
480         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
481 rally
482         intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept
483 but flagged."""
484         - with open(manifest_path) as fh:
485         -     manifest = json.load(fh)
486         + manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
487         out: List[Dict[str, Any]] = []
488         for v in manifest:
489             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
490 diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
491 index 2ea9483..b017f94 100644
492 --- a/backend/eval/rally_seq_proto.py
493 +++ b/backend/eval/rally_seq_proto.py
494 @@ -29,7 +29,6 @@ Run:
495     from __future__ import annotations
496
497     import argparse
498     -import json
499     from typing import Dict, List, Tuple
500
501     import numpy as np

```

```

500 @@ -38,6 +37,7 @@ from scipy.stats import rankdata
501
502     from backend.eval.calibrate_wasb import load_golden_gts
503     from backend.eval.classifier import LogisticRegression
504 +from backend.eval.golden_manifest import load_resolved_manifest
505     from backend.eval.metrics import score
506
507     # Feature extraction is single-sourced in backend/features (ADR-008): training and
508 @@ -196,8 +196,7 @@ def main() -> None:
509         help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
510     )
511     args = ap.parse_args()
512 -    with open(args.manifest, encoding="utf-8") as f:
513 -        run(json.load(f))
514 +    run(load_resolved_manifest(args.manifest))
515
516
517     if __name__ == "__main__":
518 diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
519 index 328857b..0f1b1f4 100644
520 --- a/backend/eval/serve_contrast.py
521 +++ b/backend/eval/serve_contrast.py
522 @@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
523     from backend.eval import windowing as W
524     from backend.eval.calibrate_wasb import load_golden_gts
525     from backend.eval.gemini_refine import merge_overlaps
526 +from backend.eval.golden_manifest import load_resolved_manifest
527     from backend.eval.metrics import score
528     from backend.eval.promotion import promote_if_served_safe
529     from backend.pipeline.detectors import rally_gate
530 @@ -194,8 +195,7 @@ def main() -> None:
531     )
532     args = ap.parse_args()
533
534 -    with open(args.manifest, encoding="utf-8") as f:
535 -        specs = json.load(f)
536 +    specs = load_resolved_manifest(args.manifest)
537     result = run_contrast(
538         specs,
539         iou=args.iou,
540 diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
541 index 994b9c1..c2151fb 100644
542 --- a/backend/eval/served_gate_a.py
543 +++ b/backend/eval/served_gate_a.py
544 @@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
545     from __future__ import annotations
546
547     import argparse
548 -import json
549     from dataclasses import dataclass
550     from typing import Any, Dict, List, Optional, Tuple
551
552     from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
553     from backend.eval import segmentation_metrics as _segm
554     from backend.eval.calibrate_wasb import load_golden_gts
555 +from backend.eval.golden_manifest import load_resolved_manifest
556     from backend.eval.metrics import Interval, Score, score
557     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
558     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
559 @@ -196,8 +196,7 @@ def format_table(
560     def run(
561         manifest_path: str, iou: float = 0.5
562     ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
563 -    with open(manifest_path, encoding="utf-8") as f:
564 -        specs = json.load(f)

```

```

565     + specs = load_resolved_manifest(manifest_path)
566     results = evaluate_manifest(specs, iou=iou)
567     return results, aggregate(results)
568
569     diff --git a/docs/ALPHA_LAUNCH_READINESS.md b/docs/ALPHA_LAUNCH_READINESS.md
570     index 04d46d2..bcc7fea 100644
571     --- a/docs/ALPHA_LAUNCH_READINESS.md
572     +++ b/docs/ALPHA_LAUNCH_READINESS.md
573     @@ -96,7 +96,7 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE
= shipped/
574     |----|-----|-----|-----|-----|-----|----|
575     | A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-
indexer` | - | No | DONE | #451 |
576     | A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check,
edge auth ON, upload -> process -> jobs -> compile -> download verified | `khelsutra` +
`badminton-highlight-indexer` | A0 | Owner spend / CF config | TODO | - |
577     -| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | TODO | - |
578     +| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | IN PROGRESS | this PR |
579     | A3 | Promote the 15-video source-video/MD5 records into the collector/vault path;
reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-
collector` + vault | A2 | Owner upload/storage scope | TODO | - |
580     | A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
581     | A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6`
floor for future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |
582     @@ -118,6 +118,15 @@ Legend: TODO = not started, IN PROGRESS = actively being worked,
DONE = shipped/
583     5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
584     6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy
limits, and launch checklist.
585
586     +A2 smoke command:
587     +
588     +```powershell
589     +python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15
590     +python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15 --write output/human_lovo_manifest.local.json
591     +```
592     +
593     +The checker resolves stale absolute paths against the repo's sibling `Annotation Setup`
folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is
absent.
594     +
595     ## 9. Open questions
596
597     1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or
should P10/P11 be reopened before any tester wave?
598     diff --git a/docs/data_pipeline/GOLDEN_VIDEOS.md b/docs/data_pipeline/GOLDEN_VIDEOS.md
599     index 6d68eeb..bb197bd 100644
600     --- a/docs/data_pipeline/GOLDEN_VIDEOS.md
601     +++ b/docs/data_pipeline/GOLDEN_VIDEOS.md
602     @@ -22,6 +22,11 @@ auditable. Updated whenever a new video is labeled + ingested.
603     - **Ingestion** = copy the trajectory + labels to durable paths, write
`output/<name>_golden_features.json`
604     (`gts` + candidates), append a row to `output/human_lovo_manifest.json`.
(`scratch/at_par.py` scores a
605     single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole
corpus.)

```

```

606     +- **Path smoke check:** before running corpus evals, run
607     + `python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15`.
608     + The checker rebases stale absolute paths against sibling `Annotation Setup` folders
and `output/`, and
609     + exits non-zero if any required trajectory or labels file is missing. Add `--write
output/human_lovo_manifest.local.json`
610     + when a resolved local manifest artifact is useful.
611     - **Guardrails:** owned footage only; **minors excluded**; labels never derived from a
paid LLM (Gemini is
612         inference/reference only). Commercial-clean.
613
614     @@ -57,4 +62,5 @@ auditable. Updated whenever a new video is labeled + ingested.
615     ## How to isolate the golden videos
616     Each row's **source proxy** column is the file to copy into a dedicated golden-videos
location. The labels
617     live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
618     -trajectory is in `Collect/Trajectories/` (path in the manifest). Keep this table in
sync on every new ingest.
619     +trajectory is in `Annotation Setup/Trajectories/` (older manifests may still say
`Collect/Trajectories/`;
620     +use the path smoke checker above). Keep this table in sync on every new ingest.
621     diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
622     index d3477c8..f042eea 100644
623     --- a/tests/test_fusion_compare.py
624     +++ b/tests/test_fusion_compare.py
625     @@ -203,6 +203,8 @@ def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
626
627         out_dir = str(tmp_path / "out")
628         os.makedirs(out_dir)
629     +     for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
630     +         (tmp_path / name).write_text("x\n", encoding="utf-8")
631         manifest = [
632             {
633                 "name": "alpha",
634     @@ -267,6 +269,8 @@ def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
635         os.makedirs(out_dir)
636         (tmp_path / "big.csv").write_text("x" * 2000)
637         (tmp_path / "small.csv").write_text("x" * 20)
638     +     (tmp_path / "b.rallies.csv").write_text("x\n", encoding="utf-8")
639     +     (tmp_path / "s.rallies.csv").write_text("x\n", encoding="utf-8")
640         manifest = [
641             {
642                 "name": "big",
643     diff --git a/tests/test_fusion_golden_telemetry.py
b/tests/test_fusion_golden_telemetry.py
644     index 1fab90..bdefc88 100644
645     --- a/tests/test_fusion_golden_telemetry.py
646     +++ b/tests/test_fusion_golden_telemetry.py
647     @@ -45,6 +45,9 @@ def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
648
649
650     def _write_manifest(tmp_path, names):
651     +     for n in names:
652     +         (tmp_path / f"{n}.csv").write_text("x\n", encoding="utf-8")
653     +         (tmp_path / f"{n}.rallies.csv").write_text("x\n", encoding="utf-8")
654         manifest = [
655             {
656                 "name": n,
657     diff --git a/tests/test_golden_manifest.py b/tests/test_golden_manifest.py
658     new file mode 100644
659     index 0000000..59ce1e5
660     --- /dev/null
661     +++ b/tests/test_golden_manifest.py

```

```

662 @@ -0,0 +1,158 @@
663 +import json
664 +from pathlib import Path
665 +
666 +import pytest
667 +
668 +from backend.eval import golden_manifest as gm
669 +
670 +
671 +def _write(path: Path, body: str = "x\n") -> Path:
672 +    path.parent.mkdir(parents=True, exist_ok=True)
673 +    path.write_text(body, encoding="utf-8")
674 +    return path
675 +
676 +
677 +def _write_manifest(repo: Path, rows: list[dict]) -> Path:
678 +    path = repo / "output" / "human_lovo_manifest.json"
679 +    path.parent.mkdir(parents=True, exist_ok=True)
680 +    path.write_text(json.dumps(rows), encoding="utf-8")
681 +    return path
682 +
683 +
684 +def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
685 +    workspace = tmp_path / "workspace"
686 +    repo = workspace / "badminton-highlight-indexer"
687 +    repo.mkdir(parents=True)
688 +    traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
689 +    old_label = _write(
690 +        workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
691 +    )
692 +    output_label = _write(repo / "output" / "Bar.rallies.csv")
693 +    manifest = _write_manifest(
694 +        repo,
695 +        [
696 +            {
697 +                "name": "foo",
698 +                "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
699 +                "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
700 +                "fps": 30,
701 +                "frame_width": 1920,
702 +            },
703 +            {
704 +                "name": "bar",
705 +                "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
706 +                "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
707 +                "fps": 30,
708 +                "frame_width": 1920,
709 +            },
710 +        ],
711 +    )
712 +
713 +    specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
714 +
715 +    assert issues == []
716 +    assert Path(specs[0]["trajectory"]) == traj.resolve()
717 +    assert Path(specs[0]["labels"]) == old_label.resolve()
718 +    assert Path(specs[1]["labels"]) == output_label.resolve()
719 +
720 +
721 +def test_strict_loader_reports_missing_required_files(tmp_path):
722 +    repo = tmp_path / "repo"
723 +    repo.mkdir()

```

```

724 +     manifest = _write_manifest(
725 +         repo,
726 +         [
727 +             {
728 +                 "name": "missing",
729 +                 "trajectory": "C:/old/missing_traj.csv",
730 +                 "labels": "C:/old/missing.rallies.csv",
731 +             }
732 +         ],
733 +     )
734 +
735 +     with pytest.raises(gm.ManifestPathError) as excinfo:
736 +         gm.load_resolved_manifest(manifest, repo_root_path=repo)
737 +
738 +     assert len(excinfo.value.issues) == 2
739 +     assert "file not found" in str(excinfo.value)
740 +
741 +
742 +def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
743 +     workspace = tmp_path / "workspace"
744 +     repo = workspace / "badminton-highlight-indexer"
745 +     repo.mkdir(parents=True)
746 +     traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
747 +     manifest = _write_manifest(
748 +         repo,
749 +         [
750 +             {
751 +                 "name": "foo",
752 +                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
753 +                 "labels": "C:/old/missing.rallies.csv",
754 +             }
755 +         ],
756 +     )
757 +
758 +     specs, issues = gm.resolve_manifest_paths(
759 +         manifest, repo_root_path=repo, required_fields=("trajectory",)
760 +     )
761 +
762 +     assert issues == []
763 +     assert Path(specs[0]["trajectory"]) == traj.resolve()
764 +     assert specs[0]["labels"] == "C:/old/missing.rallies.csv"
765 +
766 +
767 +def test_cli_writes_resolved_manifest(tmp_path, capsys):
768 +     workspace = tmp_path / "workspace"
769 +     repo = workspace / "badminton-highlight-indexer"
770 +     repo.mkdir(parents=True)
771 +     _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
772 +     _write(repo / "output" / "Foo.rallies.csv")
773 +     manifest = _write_manifest(
774 +         repo,
775 +         [
776 +             {
777 +                 "name": "foo",
778 +                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
779 +                 "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
780 +             }
781 +         ],
782 +     )
783 +     out = repo / "output" / "human_lovo_manifest.local.json"
784 +
785 +     rc = gm.main(
786 +         [
787 +             "--manifest",
788 +             str(manifest),

```

```

789 +         "--repo-root",
790 +         str(repo),
791 +         "--min-count",
792 +         "1",
793 +         "--write",
794 +         str(out),
795 +     ]
796 + )
797 +
798 + captured = capsys.readouterr()
799 + assert rc == 0
800 + assert "OK:" in captured.out
801 + assert out.is_file()
802 + resolved = json.loads(out.read_text(encoding="utf-8"))
803 + assert Path(resolved[0]["trajectory"]).is_file()
804 + assert Path(resolved[0]["labels"]).is_file()
805 +
806 +
807 +def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
808 +     repo = tmp_path / "repo"
809 +     repo.mkdir()
810 +     manifest = _write_manifest(
811 +         repo,
812 +         [{"name": "foo", "trajectory": "missing.csv", "labels":
"missing.rallies.csv"}],
813 +     )
814 +
815 +     rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
816 +
817 +     captured = capsys.readouterr()
818 +     assert rc == 2
819 +     assert "golden manifest path check failed" in captured.err
820 +     assert "foo: trajectory" in captured.err
821 +     diff --git a/tests/test_served_gate_a_regression.py
b/tests/test_served_gate_a_regression.py
822 +     index 7fd7630..56848df 100644
823 +     --- a/tests/test_served_gate_a_regression.py
824 +     +++ b/tests/test_served_gate_a_regression.py
825 +     @@ -37,7 +37,6 @@ re-measurement as the corpus grows.
826 +
827 +     from __future__ import annotations
828 +
829 +     -import json
830 +     import os
831 +
832 +     import pytest
833 +     @@ -49,6 +48,10 @@ pytest.importorskip("cv2")
834 +     from backend.eval import segmentation_metrics as _segm # noqa: E402
835 +     from backend.eval import served_gate_a as sga # noqa: E402
836 +     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
837 +     +from backend.eval.golden_manifest import ( # noqa: E402
838 +         ManifestPathError,
839 +         load_resolved_manifest,
840 +     )
841 +     from backend.eval.metrics import score # noqa: E402
842 +     from backend.pipeline.detectors import rally_gate # noqa: E402
843 +     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
844 +     @@ -64,18 +67,14 @@ MANIFEST = os.environ.get(
845 +
846 +
847 +     def _golden_specs():
848 +         - """Return the manifest specs whose trajectory + label files are all present, else
849 +         + """Return rebased manifest specs when trajectory + label files are present, else
850 +         + """

```



```

850         if not os.path.isfile(MANIFEST):
851             return []
852     - with open(MANIFEST, encoding="utf-8") as f:
853     -     specs = json.load(f)
854     -     ok = [
855     -         s
856     -         for s in specs
857     -         if os.path.isfile(str(s.get("trajectory", "")))
858     -         and os.path.isfile(str(s.get("labels", "")))
859     -     ]
860     -     return ok if len(ok) >= 6 else []
861     +     try:
862     +         specs = load_resolved_manifest(MANIFEST)
863     +     except ManifestPathError:
864     +         return []
865     +     return specs if len(specs) >= 6 else []
866
867
868     _SPECS = _golden_specs()
869 diff --git a/training/gen0/harness.py b/training/gen0/harness.py
870 index 6eec29f..433d2cb 100644
871 --- a/training/gen0/harness.py
872 +++ b/training/gen0/harness.py
873 @@ -36,6 +36,7 @@ from typing import Dict, List, Optional
874     import numpy as np
875
876     from backend.eval.classifier import LogisticRegression
877 +from backend.eval.golden_manifest import load_resolved_manifest
878     from backend.eval.gt_loader import load_gt_intervals
879     from backend.eval.rally_seq_proto import (
880         _seg_f1,
881 @@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
882     def run(
883         manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
884     ) -> dict:
885     -     with open(manifest_path, encoding="utf-8") as f:
886     -         specs = json.load(f)
887     +     specs = load_resolved_manifest(manifest_path)
888         floor = None
889         if floor_path:
890             with open(floor_path, encoding="utf-8") as f:
891

```

7. assistant (2026-07-03T17:33:20.199Z)

Now let me read the complete golden_manifest.py file and examine specific areas of concern:

8. user (2026-07-03T17:33:20.757Z)

```

1     """Portable golden-corpus manifest loader and smoke checker.
2
3     The owner-side golden manifest is intentionally gitignored and has historically
4     carried absolute paths from whichever box ingested a clip. This module keeps that
5     artifact usable after the workspace moves by resolving stale paths against the
6     current repo layout and failing loudly when required corpus files are absent.
7
8     Default local layout supported:
9     - ../Annotation Setup/Trajectories/<trajectory>.csv
10    - ../Annotation Setup/Golden Labelled/<labels>.csv
11    - output/<labels>.csv
12
13    Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14    """
15
16    from __future__ import annotations

```

```

17
18 import argparse
19 import json
20 import os
21 import sys
22 from dataclasses import dataclass
23 from pathlib import Path, PureWindowsPath
24 from typing import Any, Iterable, Sequence
25
26 DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27 SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28 DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30 _FIELD_DIR = {
31     "trajectory": "Trajectories",
32     "labels": "Golden Labelled",
33 }
34
35
36 @dataclass(frozen=True)
37 class ManifestIssue:
38     """One manifest path problem found by the resolver."""
39
40     name: str
41     field: str
42     path: str
43     reason: str
44
45     def message(self) -> str:
46         shown = f" {self.path!r}" if self.path else ""
47         return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50 class ManifestPathError(RuntimeError):
51     """Raised when a strict manifest load finds missing required files."""
52
53     def __init__(self, manifest_path: str | os.PathLike[str], issues:
54 Sequence[ManifestIssue]):
55         self.manifest_path = os.fspath(manifest_path)
56         self.issues = list(issues)
57         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
58         more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12} more"
59         super().__init__(
60             f"golden manifest path check failed for {self.manifest_path}: "
61             f"{len(self.issues)} issue(s)\n{detail}{more}"
62         )
63
64 def repo_root() -> Path:
65     """Return the repository root for this module."""
66
67     return Path(__file__).resolve().parents[2]
68
69
70 def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
71     """Load the flat golden manifest list and validate its outer shape."""
72
73     path = Path(manifest_path)
74     with path.open(encoding="utf-8") as f:
75         data = json.load(f)
76     if not isinstance(data, list):
77         raise ValueError(f"golden manifest must be a JSON list: {path}")
78     out: list[dict[str, Any]] = []
79     for i, row in enumerate(data):
80         if not isinstance(row, dict):

```

```

81         raise ValueError(f"golden manifest row {i} is not an object: {path}")
82     out.append(dict(row))
83     return out
84
85
86 def resolve_manifest_paths(
87     manifest_path: str | os.PathLike[str],
88     *,
89     repo_root_path: str | os.PathLike[str] | None = None,
90     extra_roots: Iterable[str | os.PathLike[str]] = (),
91     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93     """Load and rebase required path fields in a golden manifest.
94
95     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97     existing local path. Missing required fields are reported in ``issues``.
98     """
99
100     manifest = load_manifest(manifest_path)
101     return resolve_manifest_specs(
102         manifest,
103         manifest_path,
104         repo_root_path=repo_root_path,
105         extra_roots=extra_roots,
106         required_fields=required_fields,
107     )
108
109
110 def load_resolved_manifest(
111     manifest_path: str | os.PathLike[str],
112     *,
113     repo_root_path: str | os.PathLike[str] | None = None,
114     extra_roots: Iterable[str | os.PathLike[str]] = (),
115     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116     strict: bool = True,
117 ) -> list[dict[str, Any]]:
118     """Load a manifest with stale paths rebased; optionally raise on missing files."""
119
120     specs, issues = resolve_manifest_paths(
121         manifest_path,
122         repo_root_path=repo_root_path,
123         extra_roots=extra_roots,
124         required_fields=required_fields,
125     )
126     if strict and issues:
127         raise ManifestPathError(manifest_path, issues)
128     return specs
129
130
131 def resolve_manifest_specs(
132     specs: Sequence[dict[str, Any]],
133     manifest_path: str | os.PathLike[str],
134     *,
135     repo_root_path: str | os.PathLike[str] | None = None,
136     extra_roots: Iterable[str | os.PathLike[str]] = (),
137     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139     """Resolve an already-loaded manifest list."""
140
141     manifest = Path(manifest_path)
142     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143     roots = [Path(p) for p in _env_roots()]
144     roots.extend(Path(p) for p in extra_roots)
145     required = tuple(required_fields)

```

```

146
147     resolved: list[dict[str, Any]] = []
148     issues: list[ManifestIssue] = []
149     for idx, spec in enumerate(specs):
150         row = dict(spec)
151         name = str(row.get("name") or f"row[{idx}]")
152         for field in required:
153             raw = row.get(field)
154             if not raw:
155                 issues.append(ManifestIssue(name, field, "", "required field missing"))
156                 continue
157             hit = resolve_existing_path(
158                 str(raw),
159                 field,
160                 manifest_path=manifest,
161                 repo_root_path=root,
162                 extra_roots=roots,
163             )
164             if hit is None:
165                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166             else:
167                 row[field] = str(hit)
168             resolved.append(row)
169     return resolved, issues
170
171
172 def resolve_existing_path(
173     raw_path: str,
174     field: str,
175     *,
176     manifest_path: str | os.PathLike[str],
177     repo_root_path: str | os.PathLike[str],
178     extra_roots: Iterable[str | os.PathLike[str]] = (),
179 ) -> Path | None:
180     """Resolve one manifest path field to an existing file, if possible."""
181
182     for candidate in _candidate_paths(
183         raw_path,
184         field,
185         Path(manifest_path),
186         Path(repo_root_path),
187         [Path(p) for p in extra_roots],
188     ):
189         if candidate.is_file():
190             return candidate.resolve()
191     return None
192
193
194 def _env_roots() -> list[Path]:
195     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
196     if not raw:
197         return []
198     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
199
200
201 def _basename(raw_path: str) -> str:
202     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203     win_name = PureWindowsPath(raw_path).name
204     return win_name or Path(raw_path).name
205
206
207 def _candidate_paths(
208     raw_path: str,
209     field: str,
210     manifest_path: Path,

```

```

211     root: Path,
212     extra_roots: Sequence[Path],
213 ) -> list[Path]:
214     basename = _basename(raw_path)
215     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216     manifest_dir = manifest_path.resolve().parent
217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]
239         )
240     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241     return _dedup_paths(dirs)
242
243
244 def _expand_root(root: Path, field: str) -> list[Path]:
245     field_dir = _FIELD_DIR.get(field)
246     dirs = [root, root / "output"]
247     if field_dir:
248         dirs.extend(
249             [
250                 root / field_dir,
251                 root / "Annotation Setup" / field_dir,
252                 root / "Annotation Setup" / "Collect" / field_dir,
253             ]
254         )
255     return dirs
256
257
258 def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259     out: list[Path] = []
260     seen: set[str] = set()
261     for path in paths:
262         try:
263             key = os.path.normcase(os.path.abspath(os.fspath(path)))
264         except OSError:
265             key = os.fspath(path)
266         if key in seen:
267             continue
268         seen.add(key)
269         out.append(path)
270     return out
271
272
273 def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274     """Write a resolved manifest JSON atomically enough for local tooling."""

```

```

275
276 out = Path(path)
277 out.parent.mkdir(parents=True, exist_ok=True)
278 tmp = out.with_name(out.name + ".tmp")
279 with tmp.open("w", encoding="utf-8") as f:
280     json.dump(list(specs), f, indent=2)
281     f.write("\n")
282 os.replace(tmp, out)
283
284
285 def build_parser() -> argparse.ArgumentParser:
286     p = argparse.ArgumentParser(
287         description="Check/rebase the local golden corpus manifest paths."
288     )
289     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
290     p.add_argument(
291         "--repo-root",
292         default=None,
293         help="repo root used for default sibling output/Annotation Setup lookups",
294     )
295     p.add_argument(
296         "--root",
297         action="append",
298         default=[],
299         help=(
300             "additional search root; repeatable. Also accepts roots via "
301             f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
302         ),
303     )
304     p.add_argument(
305         "--require",
306         action="append",
307         choices=sorted(_FIELD_DIR),
308         default=None,
309         help="path field to require; repeatable. Defaults to trajectory and labels.",
310     )
311     p.add_argument(
312         "--min-count",
313         type=int,
314         default=None,
315         help="fail unless at least this many manifest rows are present",
316     )
317     p.add_argument(
318         "--write",
319         default=None,
320         help="optional path to write the resolved local manifest JSON",
321     )
322     return p
323
324
325 def main(argv: Sequence[str] | None = None) -> int:
326     args = build_parser().parse_args(argv)
327     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328     try:
329         original = load_manifest(args.manifest)
330         resolved, issues = resolve_manifest_specs(
331             original,
332             args.manifest,
333             repo_root_path=args.repo_root,
334             extra_roots=args.root,
335             required_fields=required,
336         )
337     except (OSError, ValueError, json.JSONDecodeError) as e:
338         print(f"ERROR: {e}", file=sys.stderr)
339         return 2

```

```

340
341     if args.min_count is not None and len(resolved) < args.min_count:
342         issues.append(
343             ManifestIssue(
344                 "manifest",
345                 "rows",
346                 str(len(resolved)),
347                 f"expected at least {args.min_count} row(s)",
348             )
349         )
350
351     if issues:
352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377 if __name__ == "__main__":
378     raise SystemExit(main())
379

```

9. assistant (2026-07-03T17:33:22.783Z)

Now let me check the consumer files to see if they still use `json` elsewhere:

10. user (2026-07-03T17:33:24.181Z)

grep: C:\Users\avidu\Projects\kshelsutra-gurubadminton-highlight-indexerbackendevalrally_seg_eval.py:
No such file or directory

11. user (2026-07-03T17:33:26.306Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2     over-segmentation-sensitive metric set, honestly.
3
4     This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` §4.5 /
§7)
5     measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6     IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7     segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).

```

```

8
9     For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10     windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11     operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15     GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     ""
21
22     from __future__ import annotations
23
24     import argparse
25     import json
26     import os
27     from typing import Any, Dict, List, Optional, Tuple
28
29     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
30     from backend.eval.golden_manifest import load_resolved_manifest
31     from backend.eval.metrics import Interval
32     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
33
34     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
35     DEFAULT_FEATURES_DIR = "output"
36     DEFAULT_IOU = 0.5
37
38     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
39     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
40     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
41     #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
42     #: as the default split point for the stratified report (``--long-tau`` overrides it).
DURATION, not
43     #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path
reports it as
44     #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
45     LONG_RALLY_TAU = 5.0
46
47     #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
48     #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the
AUDITABLE source
49     #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the
clips tagged
50     #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
51     #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
52     #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
53     MULTICOURT_CLIPS = frozenset(
54         {"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"}

```



```

55     )
56
57
58     def court_type_for(video: Dict[str, Any]) -> str:
59         """`multicourt` or `single` for a golden video. Honors an explicit manifest
60         `court_type` field when present, else falls back to membership in
61         `MULTICOURT_CLIPS`
62         (keyed by `name`), else `single`. """
63         ct = video.get("court_type")
64         if isinstance(ct, str) and ct:
65             return ct
66         return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
67
68     def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
69         """The long-rally lens: keep only intervals strictly longer than `min_duration`
70         seconds.
71         `min_duration <= 0` is an EXACT passthrough (the default-OFF semantics ? every
72         real rally has
73         positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
74         identically
75         to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
76         consistent. """
77         if min_duration <= 0.0:
78             return list(ivs)
79         return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
80
81     # ----- #
82     # Pure scoring core (no I/O ? unit-testable)
83     # ----- #
84     def score_intervals(
85         preds: List[Interval],
86         gts: List[Interval],
87         iou: float = DEFAULT_IOU,
88         tau_start: Optional[float] = None,
89         tau_end: Optional[float] = None,
90         min_duration: float = 0.0,
91     ) -> Dict[str, Any]:
92         """The full per-video metric row for predicted vs ground-truth rally intervals.
93
94         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
95         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
96         so a
97         merged mega-window is *visible* even when IoU-F1 isn't moved.
98
99         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
100         going forward;
101         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
102         strict-1:1
103         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
104         precision) + the
105         start/end boundary-error medians + the over-seg guard counts. See
106         docs/R2_EVAL_METRIC_DESIGN.md.
107
108         `min_duration` > 0 applies the **long-rally lens**
109         (:func:`filter_by_min_duration`): BOTH preds
110         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
111         so the whole
112         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
113         passthrough).
114         """
115         from backend.eval import tolerance_metrics as tm

```

```

107     preds = filter_by_min_duration(preds, min_duration)
108     gts = filter_by_min_duration(gts, min_duration)
109     ts = tm.TAU_START if tau_start is None else tau_start
110     te = tm.TAU_END if tau_end is None else tau_end
111     sc = metrics.score(preds, gts, iou_threshold=iou)
112     f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
113     ms = segmentation_metrics.merge_split_report(preds, gts)
114     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
115     be = tm.boundary_error_report(
116         preds, gts
117     ) # unconditional best-overlap (the R4-aiming diagnostic)
118     osr = tm.over_seg_report(preds, gts)
119     return {
120         "f1": sc.f1,
121         "precision": sc.precision,
122         "recall": sc.recall,
123         "mean_iou": sc.mean_iou,
124         "mean_start_error": sc.mean_start_error,
125         "mean_end_error": sc.mean_end_error,
126         "f1@0.1": f1k[0.1],
127         "f1@0.25": f1k[0.25],
128         "f1@0.5": f1k[0.5],
129         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
130         "merge_rate": ms["merge_rate"],
131         "split_rate": ms["split_rate"],
132         "merges": ms["merges"],
133         "splits": ms["splits"],
134         "missed": ms["missed"],
135         "spurious": ms["spurious"],
136         "num_pred": len(preds),
137         "num_gt": len(gts),
138         # --- R2 (tolerance-match) ? the task-faithful headline block ---
139         "tau_start": ts,
140         "tau_end": te,
141         "tol_f1": tol_f1,
142         "tol_precision": tol_p,
143         "tol_recall": tol_r,
144         "tol_matched": float(len(tol_matches)),
145         "dstart_abs_median": be["dstart"]["abs_median"],
146         "dend_abs_median": be["dend"]["abs_median"],
147         "split_count": osr["split_count"],
148         "merge_count": osr["merge_count"],
149     }
150
151
152     _AGG_KEYS = (
153         "f1",
154         "f1@0.25",
155         "f1@0.5",
156         "merge_rate",
157         "split_rate",
158         "segment_count_ratio",
159         "precision",
160         "recall",
161         "tol_f1",
162         "tol_precision",
163         "tol_recall",
164         "dstart_abs_median",
165         "dend_abs_median",
166     )
167
168
169     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
170         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.

```

```

171
172     Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
173     leave-one-video-out discipline (windows within a video are correlated)."""
174     out: Dict[str, Any] = {"n": len(rows)}
175     for k in _AGG_KEYS:
176         vals = [r[k] for r in rows]
177         out[k] = eval_stats.mean_with_ci(vals) if vals else None
178     return out
179
180
181     # ----- #
182     # Golden corpus I/O
183     # ----- #
184     def load_golden(
185         manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
186     ) -> List[Dict[str, Any]]:
187         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
188         intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged."""
189         manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
190         out: List[Dict[str, Any]] = []
191         for v in manifest:
192             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
193             gts: List[Interval] = []
194             if os.path.exists(feat):
195                 with open(feat) as fh:
196                     gts = [
197                         (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
198                     ]
199             entry: Dict[str, Any] = {
200                 "name": v["name"],
201                 "trajectory": v["trajectory"],
202                 "fps": float(v["fps"]),
203                 "frame_width": float(v["frame_width"]),
204                 "gts": gts,
205             }
206             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
207             # a normalized court_polygon + optional frame_height + court_height_pad from the
manifest.
208             for k in ("court_polygon", "frame_height", "court_height_pad"):
209                 if k in v:
210                     entry[k] = v[k]
211             out.append(entry)
212         return out
213
214
215     def windows_for(
216         video: Dict[str, Any], preset: WindowingPreset, min_crossings: int = 0
217     ) -> List[Interval]:
218         """Parse a video's trajectory, window it under `preset` ? predicted rally
intervals.
219
220         `min_crossings` > 0 applies the **net-crossings rally-state gate** (Gate-A,
221         `rally_gate.gate_windows`): drop windows whose shuttle crosses the net fewer than
this many
222         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
223         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
224         windowing (W1) leaves behind.
225
226         NEVER-EMPTY safeguard (mirrors `FusionSegmenter._select_for_handoff`): if the gate

```

```

would
227         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
228         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
229         """
230         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
231
232         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
233         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
234         ivs = windowing.windows_to_intervals(wins)
235         # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
236         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
prominent court ? a
237         # background/adjacent-court rally. Window-level (not point-level): a background
rally's central
238         # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
239         # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
240         poly = video.get("court_polygon")
241         if poly and ivs:
242             from backend.pipeline.detectors.court_gate import gate_intervals_to_court
243
244             fw = float(video["frame_width"])
245             fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
246             ivs = gate_intervals_to_court(
247                 ivs,
248                 points,
249                 poly,
250                 float(video["fps"]),

```

12. assistant (2026-07-03T17:33:28.521Z)

Good, `rally\_seg\_eval.py` still uses `json` on line 197. Let me check the other consumer files:

13. user (2026-07-03T17:33:29.247Z)

```

1         """The eval?serve worked example: score a cue at BOTH windowings and run the promotion
guard.
2
3         This is the reproducible proof of the foundational fix. For the Gate-A held-shuttle
guard it
4         builds candidate windows directly from each golden trajectory CSV under BOTH named
windowings
5         (:data:`backend.eval.windowing.PROPOSAL` and :data:`~backend.eval.windowing.SERVED`),
scores
6         CONTROL vs Gate-A (`net_crossings >= 2`) at each, and feeds the per-video held-out ?F1
into
7         :func:`backend.eval.promotion.promote_if_served_safe`.
8
9         It demonstrates the bug *and* the fix in one run:
10        - PROPOSAL windowing: Gate-A wins (historically **?F1 ? +0.074**, CI clears 0, 6/0
sign test).
11        - SERVED windowing: Gate-A does **not** win (historically **?F1 ? ?0.008**, real-
rally loss).
12        - The promotion guard therefore **vetoes** a promotion that would otherwise pass on
the
13        proposal numbers ? a cue cannot be promoted on a windowing production doesn't serve.
14
15        Unlike the persisted ``*_golden_features.json`` (which are PROPOSAL-windowed only), this
16        re-windows from the raw trajectory CSVs so the SERVED candidate set is genuinely
different.
17        Pure CPU, GPU-free, no Gemini, no DB. Reuses only shipped pieces (windowing, rally_gate,

```

```

18     metrics, promotion). Owner runs it against the real golden CSVs; tests exercise it on
synthetic
19     trajectories.
20
21     CLI::
22
23         python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
24     """
25
26     from __future__ import annotations
27
28     import argparse
29     import json
30     from typing import Any, Dict, List, Sequence, Tuple
31
32     from backend.eval import windowing as W
33     from backend.eval.calibrate_wasb import load_golden_gts
34     from backend.eval.gemini_refine import merge_overlaps
35     from backend.eval.golden_manifest import load_resolved_manifest
36     from backend.eval.metrics import score
37     from backend.eval.promotion import promote_if_served_safe
38     from backend.pipeline.detectors import rally_gate
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
40
41     Interval = Tuple[float, float]
42     IOU = 0.5
43     MIN_CROSSINGS = 2
44
45
46     def gate_a_scores_at(
47         points: List[Any],
48         gts: List[Interval],
49         fps: float,
50         frame_width: float,

```

14. assistant (2026-07-03T17:33:31.220Z)

Let me check if serve_contrast still uses json anywhere:

15. user (2026-07-03T17:33:32.154Z)

```
209:         json.dump(result, f, indent=2)
```

16. assistant (2026-07-03T17:33:34.130Z)

Good, `serve\_contrast.py` still uses `json`. Let me check the test file for the served_gate_a_regression:

17. user (2026-07-03T17:33:34.448Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4     `min_crossings=2`) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5     MagicMock runner). The offline ablation litmus
(`test_ablation.test_litmus_reproduces_gate_a`)
6     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
7     (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
8     runs the real ``FusionSegmenter`` decision seams (`_enrich_candidate_stats` net-
crossings +
9     `_select_for_handoff` Gate A) over windows from the **production**

```

```

10     ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
11     the served path's measured behaviour.
12
13     GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14     CI where the golden trajectories are absent; runs on the owner box where they are
present, over
15     **whatever golden clips are currently on disk** (``_golden_specs`` requires >= 6).
16
17     CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time;
this is the
18     SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has
since
19     outgrown). The original 6 *single-court* golden clips this litmus was first calibrated
on
20     (2026-06-14) have been retired from the box ? their labels are gone ? so the present
golden set is
21     the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094,
22     Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles,
largetest_doubles, gbaaddy,
23     testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a
single shuttle
24     while several courts rally at once, so ``trajectory_to_action_windows`` emits FEWER
windows than
25     there are rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A
only ever
26     DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served
?F1 ?
27     -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL
windowing the
28     same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation
(1.36?0.79).
29
30     The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
31     and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
32     (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
33     **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
34     litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
35     re-measurement as the corpus grows.
36     ""
37
38     from __future__ import annotations
39
40     import os
41
42     import pytest
43
44     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
45     # (CI), skip the whole module rather than erroring at import.
46     pytest.importorskip("cv2")
47
48     from backend.eval import segmentation_metrics as _segm # noqa: E402
49     from backend.eval import served_gate_a as sga # noqa: E402
50     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
51     from backend.eval.golden_manifest import ( # noqa: E402
52         ManifestPathError,
53         load_resolved_manifest,
54     )

```

```

55     from backend.eval.metrics import score # noqa: E402
56     from backend.pipeline.detectors import rally_gate # noqa: E402
57     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
58
59     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
60     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
61     # out-of-tree checkout.
62     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
63     MANIFEST = os.environ.get(
64         "SERVED_GATE_A_MANIFEST",
65         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
66     )
67
68
69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
[]."""
71         if not os.path.isfile(MANIFEST):
72             return []
73         try:
74             specs = load_resolved_manifest(MANIFEST)
75         except ManifestPathError:
76             return []
77         return specs if len(specs) >= 6 else []
78
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(
82         not _SPECS,
83         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
84     )
85
86
87     # ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there,
not cv2) ----
88     def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
89         """The served litmus must pass a TYPED IndexingConfig (what production passes), not
a dict ?
90         `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError
(#398). Build
91         the config + drive the full served seam path on a synthetic trajectory (no golden
data needed)."""
92         from backend.config.models import IndexingConfig
93
94         cfg = sga._served_idx_cfg(2)
95         assert isinstance(cfg, IndexingConfig)
96         assert cfg.fusion.min_crossings == 2
97         traj = tmp_path / "synth.csv"
98         rows = ["Frame,Visibility,X,Y"]
99         for i in range(120): # oscillate across the net so windows + crossings form
100             x = 200 + (i % 20) * 40

```

18. assistant (2026-07-03T17:33:37.744Z)

Good. Now let me check the critical line 447-451 in golden_manifest.py for the rebased zip alignment issue:

19. user (2026-07-03T17:33:38.555Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (440). The file has 379 lines.</system-reminder>

20. user (2026-07-03T17:33:40.636Z)

```
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377 if __name__ == "__main__":
378     raise SystemExit(main())
379
```

21. assistant (2026-07-03T17:33:42.367Z)

Let me check the test for golden_manifest more closely:

22. user (2026-07-03T17:33:42.556Z)

```
1     import json
2     from pathlib import Path
3
4     import pytest
5
6     from backend.eval import golden_manifest as gm
7
8
9     def _write(path: Path, body: str = "x\n") -> Path:
10         path.parent.mkdir(parents=True, exist_ok=True)
11         path.write_text(body, encoding="utf-8")
12         return path
13
14
15     def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16         path = repo / "output" / "human_lovo_manifest.json"
17         path.parent.mkdir(parents=True, exist_ok=True)
18         path.write_text(json.dumps(rows), encoding="utf-8")
19         return path
20
21
22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
```



```

32         repo,
33         [
34             {
35                 "name": "foo",
36                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                 "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                 "fps": 30,
39                 "frame_width": 1920,
40             },
41             {
42                 "name": "bar",
43                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                 "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                 "fps": 30,
46                 "frame_width": 1920,
47             },
48         ],
49     )
50
51     specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53     assert issues == []
54     assert Path(specs[0]["trajectory"]) == traj.resolve()
55     assert Path(specs[0]["labels"]) == old_label.resolve()
56     assert Path(specs[1]["labels"]) == output_label.resolve()
57
58
59     def test_strict_loader_reports_missing_required_files(tmp_path):
60         repo = tmp_path / "repo"
61         repo.mkdir()
62         manifest = _write_manifest(
63             repo,
64             [
65                 {
66                     "name": "missing",
67                     "trajectory": "C:/old/missing_traj.csv",
68                     "labels": "C:/old/missing.rallies.csv",
69                 }
70             ],
71         )
72
73         with pytest.raises(gm.ManifestPathError) as excinfo:
74             gm.load_resolved_manifest(manifest, repo_root_path=repo)
75
76         assert len(excinfo.value.issues) == 2
77         assert "file not found" in str(excinfo.value)
78
79
80     def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
81         workspace = tmp_path / "workspace"
82         repo = workspace / "badminton-highlight-indexer"
83         repo.mkdir(parents=True)
84         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
85         manifest = _write_manifest(
86             repo,
87             [
88                 {
89                     "name": "foo",
90                     "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
91                     "labels": "C:/old/missing.rallies.csv",
92                 }
93             ],

```

```

94         )
95
96     specs, issues = gm.resolve_manifest_paths(
97         manifest, repo_root_path=repo, required_fields=("trajectory",)
98     )
99
100    assert issues == []
101    assert Path(specs[0]["trajectory"]) == traj.resolve()
102    assert specs[0]["labels"] == "C:/old/missing.rallies.csv"
103
104
105    def test_cli_writes_resolved_manifest(tmp_path, capsys):
106        workspace = tmp_path / "workspace"
107        repo = workspace / "badminton-highlight-indexer"
108        repo.mkdir(parents=True)
109        _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
110        _write(repo / "output" / "Foo.rallies.csv")
111        manifest = _write_manifest(
112            repo,
113            [
114                {
115                    "name": "foo",
116                    "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
117                    "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
118                }
119            ],
120        )
121        out = repo / "output" / "human_lovo_manifest.local.json"
122
123        rc = gm.main(
124            [
125                "--manifest",
126                str(manifest),
127                "--repo-root",
128                str(repo),
129                "--min-count",
130                "1",
131                "--write",
132                str(out),
133            ]
134        )
135
136        captured = capsys.readouterr()
137        assert rc == 0
138        assert "OK:" in captured.out
139        assert out.is_file()
140        resolved = json.loads(out.read_text(encoding="utf-8"))
141        assert Path(resolved[0]["trajectory"]).is_file()
142        assert Path(resolved[0]["labels"]).is_file()
143
144
145    def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
146        repo = tmp_path / "repo"
147        repo.mkdir()
148        manifest = _write_manifest(
149            repo,
150            [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
151        )
152
153        rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
154
155        captured = capsys.readouterr()
156        assert rc == 2
157        assert "golden manifest path check failed" in captured.err
158        assert "foo: trajectory" in captured.err

```

23. assistant (2026-07-03T17:33:45.317Z)

Let me check the fusion_golden.py changes more carefully:

24. user (2026-07-03T17:33:45.907Z)

```

155
156     def run(
157         manifest_path: str,
158         out_dir: str,
159         frame_skip: int = 3,
160         iou: float = 0.5,
161         fresh: bool = False,
162         smallest_first: bool = False,
163     ) -> List[str]:
164         specs = load_resolved_manifest(manifest_path)
165         if smallest_first:
166             # Process small videos first for quick feedback (the slow full match lands
167             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
168             specs = sorted(
169                 specs,
170                 key=lambda s: (
171                     os.path.getsize(s["trajectory"])
172                     if os.path.isfile(s.get("trajectory", ""))
173                     else 0
174                 ),
175             )
176         os.makedirs(out_dir, exist_ok=True)
177         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
178         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
179         # break extraction, so guard the build and fall back to no telemetry on any error.
180         telem: Optional[RunTelemetry]
181         try:
182             telem = RunTelemetry(
183                 os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
184             )

```

25. assistant (2026-07-03T17:33:47.976Z)

Good. Now let me check for the json import in fusion_golden.py to verify it wasn't removed while still needed:

26. user (2026-07-03T17:33:49.039Z)

```

import argparse
import json
import os
import time

```

27. assistant (2026-07-03T17:33:51.139Z)

Good, `json` is still imported in fusion_golden. Let me check all the consumer files more systematically for json usage:

28. user (2026-07-03T17:33:52.784Z)

```

Exit code 1
=== calibrate_local.py ===

```

```
=== distill_local.py ===
=== rally_seq_proto.py ===
=== served_gate_a.py ===
```

29. assistant (2026-07-03T17:33:54.919Z)

Good?all 4 files removed `import json` and they should not need it anymore. Let me verify they don't use json elsewhere by checking the full file:

30. user (2026-07-03T17:33:55.867Z)

0

31. user (2026-07-03T17:33:58.531Z)

0

32. user (2026-07-03T17:34:02.183Z)

0

33. user (2026-07-03T17:34:04.886Z)

0

34. assistant (2026-07-03T17:34:07.801Z)

Good. Now let me check training/gen0/harness.py for json usage:

35. user (2026-07-03T17:34:08.112Z)

```
1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json      (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json     (the reviewable contract: feature-schema version,
13                                         calibration, training lineage + GT hashes,
14                                         eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission
19       (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23
24     from __future__ import annotations
25
26     import argparse
27     import hashlib
28     import json
29     import os
30     import re
31     import subprocess
```

```

32     import time
33     from math import comb
34     from typing import Dict, List, Optional
35
36     import numpy as np
37
38     from backend.eval.classifier import LogisticRegression
39     from backend.eval.golden_manifest import load_resolved_manifest
40     from backend.eval.gt_loader import load_gt_intervals
41     from backend.eval.rally_seq_proto import (
42         _seg_f1,
43         labels_for,
44     )
45     from backend.eval.rally_seq_proto import (
46         run as lovo_run,
47     )
48     from backend.eval.regression import gt_hash
49     from backend.features.rally_seq import (
50         FEAT_NAMES,

```

36. assistant (2026-07-03T17:34:10.202Z)

Good, it still imports json. Let me verify it uses json:

37. user (2026-07-03T17:34:11.322Z)

```

225:         floor = json.load(f)
244:         json.dump(payload, f, indent=2)
292:         json.dump(manifest, f, indent=2)

```

38. assistant (2026-07-03T17:34:13.863Z)

Perfect. Now let me examine critical issues. Let me focus on line 447-451 of golden_manifest.py where the zip(original, resolved) occurs:

39. user (2026-07-03T17:34:14.056Z)

```

325     def main(argv: Sequence[str] | None = None) -> int:
326         args = build_parser().parse_args(argv)
327         required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328         try:
329             original = load_manifest(args.manifest)
330             resolved, issues = resolve_manifest_specs(
331                 original,
332                 args.manifest,
333                 repo_root_path=args.repo_root,
334                 extra_roots=args.root,
335                 required_fields=required,
336             )
337         except (OSError, ValueError, json.JSONDecodeError) as e:
338             print(f"ERROR: {e}", file=sys.stderr)
339             return 2
340
341         if args.min_count is not None and len(resolved) < args.min_count:
342             issues.append(
343                 ManifestIssue(
344                     "manifest",
345                     "rows",
346                     str(len(resolved)),
347                     f"expected at least {args.min_count} row(s)",
348                 )
349             )
350
351         if issues:

```

```

352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0

```

40. assistant (2026-07-03T17:34:18.517Z)

I found a critical issue! On line 360-365, the code uses `zip(original, resolved)`, but `resolved` contains ALL rows including those with issues, while the comparison is meant to count "rebased" fields. However, the `resolve_manifest_specs` function always appends every row to `resolved` (line 168 in the function), regardless of issues. So the zip should work correctly in terms of length alignment? both `original` and `resolved` have the same length because `resolve_manifest_specs` adds one row per input spec (line 149-169 shows it appends one row per iteration).

But wait? let me trace this carefully. Looking at lines 147-169 in `resolve_manifest_specs`, I see that `resolved.append(row)` happens for EVERY spec (line 168), whether there were issues or not. So lengths should match. That's actually safe.

However, let me look more closely at the `_basename` function with edge cases:

41. user (2026-07-03T17:34:18.776Z)

```

201     def _basename(raw_path: str) -> str:
202         # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203         win_name = PureWindowsPath(raw_path).name
204         return win_name or Path(raw_path).name
205

```

42. assistant (2026-07-03T17:34:20.958Z)

Let me check the `candidate_dirs` and `candidate_paths` ordering issue more carefully:

43. user (2026-07-03T17:34:21.158Z)

```

207     def _candidate_paths(
208         raw_path: str,
209         field: str,
210         manifest_path: Path,
211         root: Path,
212         extra_roots: Sequence[Path],
213     ) -> list[Path]:
214         basename = _basename(raw_path)
215         raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216         manifest_dir = manifest_path.resolve().parent

```

```

217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]
239         )
240     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241     return _dedup_paths(dirs)
242
243
244 def _expand_root(root: Path, field: str) -> list[Path]:
245     field_dir = _FIELD_DIR.get(field)
246     dirs = [root, root / "output"]
247     if field_dir:
248         dirs.extend(
249             [
250                 root / field_dir,
251                 root / "Annotation Setup" / field_dir,

```

44. assistant (2026-07-03T17:34:25.620Z)

I found a bug! On line 240, the code does:

```

```python
dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
```

```

This should be:

```

```python
dirs.extend(_expand_root(r, field) for r in extra_roots)
```

```

OR more correctly:

```

```python
for r in extra_roots:
 dirs.extend(_expand_root(r, field))
```

```

The current code is passing a generator directly to `extend()`, which will iterate the generator and add individual generators/lists as elements, not flatten them. This is a critical bug.

Actually, let me re-read this. `\_expand\_root` returns a `list[Path]`. The expression `(root\_dirs for r in extra\_roots for root\_dirs in \_expand\_root(r, field))` is a generator that yields each `Path` from each `\_expand\_root` call. When passed to `extend()`, it should work correctly? `extend()` will consume the generator and add each `Path` one by one. So this is actually correct, just verbose.

Let me focus on other issues. Let me check line 337 for `json.JSONDecodeError`:

45. user (2026-07-03T17:34:26.774Z)

True

46. assistant (2026-07-03T17:34:29.527Z)

Good, `json.JSONDecodeError` is a subclass of `ValueError`, so catching both is redundant but not wrong. The except clause on line 337 catches `(OSError, ValueError, json.JSONDecodeError)` but since `JSONDecodeError` is a `ValueError`, this is redundant. Not a bug, just code style.

Now let me check the `write_manifest` function for atomicity issues:

47. user (2026-07-03T17:34:30.359Z)

```
273     def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274         """Write a resolved manifest JSON atomically enough for local tooling."""
275
276         out = Path(path)
277         out.parent.mkdir(parents=True, exist_ok=True)
278         tmp = out.with_name(out.name + ".tmp")
279         with tmp.open("w", encoding="utf-8") as f:
280             json.dump(list(specs), f, indent=2)
281             f.write("\n")
282         os.replace(tmp, out)
```

48. assistant (2026-07-03T17:34:33.376Z)

Good?the `write_manifest` uses the temp file + `os.replace` pattern, which is atomic on Windows. No issue there.

Now let me check the `rally_seg_eval.py` change?specifically line 484:

49. user (2026-07-03T17:34:33.668Z)

```
184     def load_golden(
185         manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
186     ) -> List[Dict[str, Any]]:
187         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
188         intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged."""
189         manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
190         out: List[Dict[str, Any]] = []
191         for v in manifest:
192             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
193             gts: List[Interval] = []
194             if os.path.exists(feat):
195                 with open(feat) as fh:
196                     gts = [
197                         (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
198                     ]
199             entry: Dict[str, Any] = {
200                 "name": v["name"],
201                 "trajectory": v["trajectory"],
202                 "fps": float(v["fps"]),
203                 "frame_width": float(v["frame_width"]),
204                 "gts": gts,
205             }
206             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
207             # a normalized court_polygon + optional frame_height + court_height_pad from the
```



```

manifest.
208         for k in ("court_polygon", "frame_height", "court_height_pad"):
209             if k in v:
210                 entry[k] = v[k]
211             out.append(entry)
212         return out
213

```

50. assistant (2026-07-03T17:34:39.254Z)

I see a potential issue! On line 189, ``rally_seg_eval.load_golden`` calls ``load_resolved_manifest(manifest_path, required_fields=("trajectory",))`` with ``required_fields=("trajectory",)`` only. This means that the ``labels`` field will NOT be checked for existence by ``load_resolved_manifest``, so missing label files won't cause an exception. The doc string on line 187-188 says "Videos with no GTs are kept but flagged" which suggests this is intentional, but the code doesn't actually check for labels in the manifest. Looking at the comment, it seems like the intent is that ``labels`` is optional here because the GT intervals come from a separate ``_golden_features.json`` file (line 192), not from the manifest's ``labels`` field.

This is actually correct behavior?the function only requires ``trajectory`` and lets ``labels`` be optional. So no bug here.

Now let me look for potential shadowing issues in the candidate path ordering. The ``_candidate_dirs`` function builds a search order. Let me trace through a concrete scenario:

1. manifest_dir (repo/output)
2. root.parent / "Annotation Setup" / field_dir (../Annotation Setup/Trajectories)
3. root.parent / "Annotation Setup" / "Collect" / field_dir (../Annotation Setup/Collect/Trajectories)
4. root / "output" (repo/output)
5. extra_roots expanded

Items 1 and 4 are the same! The ``_dedup_paths`` should catch this, but let me verify it works correctly:

51. user (2026-07-03T17:34:39.546Z)

```

258     def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259         out: list[Path] = []
260         seen: set[str] = set()
261         for path in paths:
262             try:
263                 key = os.path.normcase(os.path.abspath(os.fspath(path)))
264             except OSError:
265                 key = os.fspath(path)
266             if key in seen:
267                 continue
268             seen.add(key)
269             out.append(path)
270         return out

```

52. assistant (2026-07-03T17:34:42.669Z)

Good?``_dedup_paths`` uses ``os.path.abspath`` to resolve paths to their canonical form before deduplication, so `repo/output` and `../Annotation Setup/./output` would be deduplicated correctly.

Now let me check the test for the ``--min-count`` flag interaction with issues:

53. user (2026-07-03T17:34:43.723Z)

```

"--min-count",
"1",
"--write",

```

```

        str(out),
    ]
)

captured = capsys.readouterr()
assert rc == 0
assert "OK:" in captured.out
assert out.is_file()
resolved = json.loads(out.read_text(encoding="utf-8"))
assert Path(resolved[0]["trajectory"]).is_file()
assert Path(resolved[0]["labels"]).is_file()

```

```

def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
    repo = tmp_path / "repo"
    repo.mkdir()
    manifest = _write_manifest(
        repo,

```

54. assistant (2026-07-03T17:34:45.788Z)

I don't see a test for `--min-count` failure, which could be a gap. Let me look for edge case: what if min_count is violated?

55. user (2026-07-03T17:34:46.132Z)

```

341         if args.min_count is not None and len(resolved) < args.min_count:
342             issues.append(
343                 ManifestIssue(
344                     "manifest",
345                     "rows",
346                     str(len(resolved)),
347                     f"expected at least {args.min_count} row(s)",
348                 )
349             )
350

```

56. assistant (2026-07-03T17:34:50.293Z)

I see a potential issue! The `--min-count` check on line 341 appends to `issues` and then prints all issues and returns 2. But the code on line 351 checks `if issues:` which now includes the min_count issue. So it will properly fail. Let me verify this logic is correct: if min_count is violated, an issue is added, then the code prints "ERROR: golden manifest path check failed" and returns 2. This is correct.

Now let me check for a PureWindowsPath edge case. If `raw\_path` is a string with a trailing slash like `C:/foo/bar/` what does PureWindowsPath.name return?

Let me test this scenario more carefully by reviewing the tests:

57. user (2026-07-03T17:34:50.478Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

58. user (2026-07-03T17:34:53.043Z)

```

22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"

```

```

29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
32             repo,
33             [
34                 {
35                     "name": "foo",
36                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                     "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                     "fps": 30,
39                     "frame_width": 1920,
40                 },
41                 {
42                     "name": "bar",
43                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                     "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                     "fps": 30,
46                     "frame_width": 1920,
47                 },
48             ],
49         )
50
51         specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53         assert issues == []
54         assert Path(specs[0]["trajectory"]) == traj.resolve()
55         assert Path(specs[0]["labels"]) == old_label.resolve()
56         assert Path(specs[1]["labels"]) == output_label.resolve()

```

59. assistant (2026-07-03T17:34:56.619Z)

The test uses Windows-style paths like ``C:/old/Annotation Setup/Collect/Trajectories/foo_traj.csv`` which should be extracted with ``_basename`` to ``"foo_traj.csv"``. Let me check if there's any issue with `expandvars/expanduser`:

60. user (2026-07-03T17:34:56.834Z)

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter ``offset`` type is expected as ``number`` but provided as ``array``</tool_use_error>

61. user (2026-07-03T17:34:59.084Z)

```

214         basename = _basename(raw_path)
215         raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216         manifest_dir = manifest_path.resolve().parent
217         candidates: list[Path] = [raw]
218
219         if not raw.is_absolute():
220             candidates.extend([manifest_dir / raw, root / raw])
221
222         for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223             candidates.append(directory / basename)
224         return _dedup_paths(candidates)

```

62. assistant (2026-07-03T17:35:05.307Z)

I see an issue! On line 215, if ``raw_path`` contains environment variables like ``$HOME``, ``os.path.expandvars`` will expand them. But if the expanded path has special characters or spaces, and then on line 216 the code tries to create a `Path` from it, this should be fine.

However, there's a subtle issue on line 219: the code checks ``if not raw.is_absolute()``. But

after ``expandvars`` and ``expanduser``, a path like ``~/mydir/foo.csv`` becomes ``/home/user/mydir/foo.csv`` which IS absolute. So relative paths that don't start with ``~`` or ``$`` will be treated as relative, which is correct. But what if someone has a manifest with ``~/mydir/foo.csv``? After ``expanduser``, it becomes ``/home/user/mydir/foo.csv``, and ``is_absolute()`` returns True, so the code won't add ``manifest_dir / raw`` to candidates. But the original intent was to search alongside the manifest, so the expanded absolute path won't match. This could be a subtle issue if someone has ``~/`` in their manifest and expects it to resolve relative to the manifest location.

Actually, re-reading the logic: if ``raw_path`` is ``~/foo.csv``, then after ``expanduser`` it becomes ``/home/user/foo.csv``, and ``raw.is_absolute()`` is True. So the code will only check:

1. The expanded absolute path ``/home/user/foo.csv`` (line 217)
2. The basename ``foo.csv`` in various standard directories (lines 222-223)

It won't check ``manifest_dir / expanded_path``, but that makes sense because the path is already absolute. So this is correct behavior?expand ``~/`` to an absolute path and use that directly.

Let me now check the test for the failure case to see if it asserts exit code 2:

63. user (2026-07-03T17:35:05.554Z)

```
145     def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
146         repo = tmp_path / "repo"
147         repo.mkdir()
148         manifest = _write_manifest(
149             repo,
150             [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
151         )
152
153         rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
154
155         captured = capsys.readouterr()
156         assert rc == 2
157         assert "golden manifest path check failed" in captured.err
158         assert "foo: trajectory" in captured.err
159
```

64. assistant (2026-07-03T17:35:11.304Z)

Good?the test asserts ``rc == 2`` and checks stderr for the error message. Let me now look for an issue I suspected: when running ``main()`` without issues but with ``--min-count`` failure, does it return 2? Let me trace the code again on line 341-358:

If ``args.min_count`` is not None and ``len(resolved) < args.min_count``, the code appends an issue. Then if ``issues`` is not empty, it prints and returns 2. This is correct.

Now let me look at the tests to see if there's a test for min_count failure. Looking back at the test file, I don't see one. But there's one that uses ``--min-count 1`` with 1 row available, which should pass (`test_cli_writes_resolved_manifest``).

The missing test: what if min_count=15 but only 1 row exists? The test file doesn't seem to have this. However, the PR description says tests were added, so let me check if the test is missing or if I just need to look harder.

Actually, let me reconsider the real bugs. Let me check the `served_gate_a.py` change more carefully:

65. user (2026-07-03T17:35:11.534Z)

```
<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>
```

66. user (2026-07-03T17:35:13.975Z)

```

195
196     def run(
197         manifest_path: str, iou: float = 0.5
198     ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
199         specs = load_resolved_manifest(manifest_path)
200         results = evaluate_manifest(specs, iou=iou)
201         return results, aggregate(results)
202
203
204     def main(argv: Optional[List[str]] = None) -> None:
205         ap = argparse.ArgumentParser(
206             description="Verify served Gate-A on real footage via the production
207             FusionSegmenter path."
208         )
209         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
210         ap.add_argument("--iou", type=float, default=0.5)

```

67. assistant (2026-07-03T17:35:16.669Z)

Good, `served\_gate\_a.run` calls `load\_resolved\_manifest(manifest\_path)` with default arguments, which means it requires both `trajectory` and `labels` by default. This is correct.

Now let me look at the test_served_gate_a_regression change, specifically the `\_golden\_specs` function:

68. user (2026-07-03T17:35:17.135Z)

```

69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
71         []."""
72         if not os.path.isfile(MANIFEST):
73             return []
74         try:
75             specs = load_resolved_manifest(MANIFEST)
76         except ManifestPathError:
77             return []
78         return specs if len(specs) >= 6 else []
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(

```

69. assistant (2026-07-03T17:35:38.287Z)

Perfect! The test correctly catches `ManifestPathError` and returns `[]` if there are missing files. This is good.

Now let me look for subtle issues in the candidate ordering. Let me check if there's any scenario where a WRONG file could shadow the right one:

The search order in `\_candidate\_dirs` is:

1. manifest_dir
2. root.parent / "Annotation Setup" / field_dir
3. root.parent / "Annotation Setup" / "Collect" / field_dir
4. root / "output"
5. extra_roots

And `\_candidate\_paths` returns candidates in order: first the raw path itself (if absolute or expanded), then potentially relative path candidates, then basename+directory pairs from `\_candidate\_dirs`.

The `resolve\_existing\_path` function iterates through candidates in order and returns the FIRST one that `is\_file()`. So if a file with the same basename exists in manifest_dir, it will be found before checking the "correct" Annotation Setup directories.

This is actually a SHADOWING BUG! If someone has a file named `foo.csv` in the manifest_dir (repo/output), and also has the "correct" trajectory `foo.csv` in `../Annotation Setup/Trajectories/`, the one in manifest_dir will be found first and incorrectly used!

The fix would be to reorder the candidates or to make the search smarter (e.g., check absolute paths first, then relative within the repo, then the standard directories). But the current code has manifest_dir first in line 231.

Let me double-check this issue by looking at the candidate order more carefully:

```
```python
def _candidate_paths(...):
 basename = _basename(raw_path) # e.g., "foo.csv"
 raw = Path(os.path.expandvars(os.path.expanduser(raw_path))) # e.g., Path("C:/old/foo.csv")
 manifest_dir = manifest_path.resolve().parent # e.g., Path("/repo/output")
 candidates: list[Path] = [raw] # [Path("C:/old/foo.csv")]

 if not raw.is_absolute(): # C:/old is absolute, so skip
 candidates.extend([manifest_dir / raw, root / raw])

 for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
 candidates.append(directory / basename)
 # This appends: "/repo/output/foo.csv", "../Annotation Setup/Trajectories/foo.csv", etc.

 return _dedup_paths(candidates)
```
```

So the candidates list is:

1. Path("C:/old/foo.csv") ? the stale absolute path (unlikely to exist on new machine)
2. Path("/repo/output/foo.csv") ? manifest_dir / basename
3. Path("../Annotation Setup/Trajectories/foo.csv") ? correct location
4. etc.

When `resolve\_existing\_path` iterates through these in order, it will find the first existing file. If someone accidentally has a `foo.csv` in `repo/output`, it will be found before the correct one in `../Annotation Setup/Trajectories/foo.csv`.

This is a real shadowing bug! It should check the "standard" directories (Annotation Setup) before the manifest_dir.

Let me now prepare my findings. I've identified 1 confirmed bug (shadowing in candidate order) and need to verify if there are any other issues.

Actually, let me reconsider: is putting manifest_dir first actually wrong? The manifest is in repo/output, so if a stale path refers to a file that was originally in the same directory as the manifest, putting manifest_dir first would be correct. But the docstring says "Default local layout supported" lists specific paths like `../Annotation Setup/Trajectories/<trajectory>.csv`, suggesting that the standard layout should be preferred.

Let me check if there are any tests that would fail if we reordered:

Looking at the test `test\_resolves\_stale\_absolute\_paths\_to\_workspace\_layout`, it uses Windows-style absolute paths like `C:/old/Annotation Setup/Collect/Trajectories/foo\_traj.csv`, and the test expects these to resolve correctly even though the "correct" paths are in the sibling Annotation Setup directories. The test creates:

- `workspace/Annotation Setup/Trajectories/foo\_traj.csv`
- `workspace/Annotation Setup/Golden Labelled/Foo.rallies.csv`
- `repo/output/Bar.rallies.csv`

And manifests refer to:

- `C:/old/Annotation Setup/Collect/Trajectories/foo\_traj.csv` should resolve to the first path
- `C:/old/badminton-highlight-indexer/output/Bar.rallies.csv` should resolve to the third path

If manifest_dir is checked before the standard directories, it wouldn't find these files (since they're not in repo/output), so it would continue searching and find them in the standard

directories. So the current ordering works for the test.

But the shadowing bug still exists: if someone has a file with the same basename in both manifest_dir AND the standard directory, the one in manifest_dir will be used, which might not be the intended one.

This is a legitimate bug to report.

Let me now check if there are any other issues I've missed.

Let me look at line 362 again for the `zip(original, resolved)` issue:

```
```python
rebased = sum(
 1
 for before, after in zip(original, resolved)
 for field in required
 if str(before.get(field, "")) != str(after.get(field, ""))
)
```
```

Wait, I need to reconsider the logic here. The code counts the number of (row, field) pairs where the before and after values differ. But is this correct?

Looking at the comment in the main function, it says "number of path(s) rebased". So if a row had 2 required fields (trajectory and labels), and both were rebased, it should count as 2, not 1. The code does count each (row, field) pair separately, so this is correct.

However, there's a subtle issue: what if a row had a missing field (and thus an issue was recorded)? The code still includes that row in `resolved` (line 168), so the before/after comparison will include it. If the field was missing in `before`, then `before.get(field, "")` returns `""`, and `after.get(field, "")` also returns `""` (assuming the field wasn't found and the issue was recorded). So they would be equal and not counted as rebased. This is correct behavior.

Actually wait, let me re-read resolve_manifest_specs. If a field is missing (raw is falsy), then line 155-156 records an issue but then `continue`, skipping the `row[field] = str(hit)` assignment. So the row is never updated with a resolved path if the field was missing or not found. This means:

```
- original: {"trajectory": "missing.csv"}
- resolved: {"trajectory": "missing.csv"} (unchanged because it wasn't found)
```

So the rebased count won't include missing files, which makes sense? they're reported as issues, not as rebased paths.

Now let me check one more potential issue: what if the manifest JSON is valid JSON but doesn't match the expected schema (e.g., a row doesn't have a "name" field)? Let me check the code:

```
Line 151: `name = str(row.get("name")) or f"row[{idx}]`
```

So if a row doesn't have a "name", it uses `row[0]` as the name in error messages. This is good error handling.

Let me now compile my findings:

1. **Shadowing bug in candidate path order (line 231)**:** manifest_dir is checked first, which could shadow the correct file in the standard Annotation Setup directories if a file with the same basename exists in both locations.
2. Let me check if there are any other issues I've missed by looking at edge cases in _basename:

70. user (2026-07-03T17:35:41.025Z)

| | | | |
|------------------|-------------------|------------------|---------|
| 'C:/foo/bar.csv' | -> win: 'bar.csv' | posix: 'bar.csv' | result: |
| 'bar.csv' | | | |

| | | | |
|---------------------|--------------------|-------------------|---------|
| 'C:/foo/bar/' | -> win: 'bar' | posix: 'bar' | result: |
| 'bar' | | | |
| '' | -> win: '' | posix: '' | result: |
| '' | | | |
| '/' | -> win: '' | posix: '' | result: |
| '' | | | |
| 'C:/' | -> win: '' | posix: '' | result: |
| '' | | | |
| 'relative/path.csv' | -> win: 'path.csv' | posix: 'path.csv' | result: |
| 'path.csv' | | | |
| './path.csv' | -> win: 'path.csv' | posix: 'path.csv' | result: |
| 'path.csv' | | | |

71. assistant (2026-07-03T17:35:50.171Z)

Interesting! If the `raw_path` is a directory with a trailing slash (like `"C:/foo/bar/"`), then `_basename` returns "bar" instead of the actual filename. This could cause a file to not be found if the manifest has a directory path instead of a file path.`

However, this is probably a data quality issue (the manifest should have file paths, not directory paths), and the code would correctly report it as "file not found" since ``bar/basename` wouldn't match anything. So this is not a code bug per se, just a data validation issue.`

Now let me look for issues in the test coverage. Looking at the test file, I see tests for:

1. Resolving stale absolute paths ?
2. Strict loader reporting missing files ?
3. Trajectory-only resolution ?
4. CLI writing resolved manifest ?
5. CLI failing on missing file ?

Missing test: what if `--min-count` is violated but no other issues exist? There's no explicit test for this scenario. Let me check if the code would handle it correctly anyway:`

Actually, looking again at the code and test, I think the logic is correct. If `min_count` is violated, an issue is appended and then the code returns 2. The test `test_cli_fails_loudly_on_missing_file` already asserts `rc == 2`, so that pattern is tested.`

Let me look for one more potential issue: what if ``RALLY_GOLDEN_SEARCH_ROOTS` contains an invalid path like a relative path or a non-existent path? The code would still include it in the search, and `_candidate_dirs` would return it. Then when resolving paths, it would check `directory / basename` for each directory, and if the directory doesn't exist, `is_file()` would return False. So this is handled correctly?invalid roots are silently skipped.`

Let me finalize my findings:

****CANDIDATES FOR REVIEW:****

1. ****Line 231 (potential shadowing bug)**:** ``manifest_dir` is checked first in `_candidate_dirs`, which could shadow the correct file in standard Annotation Setup directories if a file with the same basename exists in both locations.`

2. ****Line 337 (redundant exception catching)**:** ``json.JSONDecodeError` is caught alongside `ValueError`, but JSONDecodeError` is a subclass of ValueError, so catching both is redundant (though not a bug).`

3. Let me check if there's an issue with how the `rally_seg_eval` consumer handles labels. Let me re-read that function:`

Actually, looking at line 189 again: ``load_resolved_manifest(manifest_path, required_fields=("trajectory",))` means labels are optional. This is correct for this use case because the labels come from `_golden_features.json`, not from the manifest.`

But wait?let me check if other consumers expect labels to be present. Let me check `calibrate_local.py:`

72. user (2026-07-03T17:35:50.935Z)

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

73. user (2026-07-03T17:35:53.185Z)

```
155     def main() -> None:
156         ap = argparse.ArgumentParser(
157             description="Calibrate the local-only pipeline against Gemini silver labels."
158         )
159         ap.add_argument(
160             "--manifest",
161             required=True,
162             help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]",
163         )
164         ap.add_argument("--iou", type=float, default=0.5)
165         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
166         args = ap.parse_args()
167         specs = load_resolved_manifest(args.manifest)
168         run(specs, iou=args.iou, metric=args.metric)
169
170
171     if __name__ == "__main__":
172         main()
173
```

74. assistant (2026-07-03T17:35:55.813Z)

On line 167, `calibrate\_local` calls `load\_resolved\_manifest(args.manifest)` with default arguments, which requires both trajectory AND labels. Let me check the run function to see if it uses both:

75. user (2026-07-03T17:35:56.186Z)

<tool_use_error>InputValidationError: Read was called with input that could not be parsed as JSON.
You sent (first 154 of 154 bytes): {"file_path": "C:\\Users\\avidu\\Projects\\khelsutra-guru\\badminton-highlight-indexer\\backend\\eval\\calibrate_local.py", "offset": [95, "limit": [130]]
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

76. user (2026-07-03T17:35:59.104Z)

```
95         {"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts": gts}
96     )
97     return videos
98
99
100     def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
101         videos = build_videos(specs)
102         grid = local_grid()
103         print(
104             f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
105             f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ==="
106         )
107
108         # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
109         video).
110         per_video = []
111         for v in videos:
112             best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
113             imp = improvement_report(
```

```

113         v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"], iou
114     )
115     b, t = imp["baseline"], imp["tuned"]
116     print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
117     print(
118         f"    baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}"
119     )
120     print(
121         f"    best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f} "
122         f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)"
123     )
124     per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})
125
126     # Honest cross-video: pick config on the OTHER videos, score the held-out one.
127     lovo = leave_one_video_out(videos, grid, metric, iou)
128     # Baseline's held-out score (no tuning) for the honest before/after.
129     base_held = [
130         evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in videos
131     ]
132     base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
133
134     print("\n--- HONEST leave-one-video-out (the number to trust) ---")

```

77. assistant (2026-07-03T17:36:01.194Z)

Let me check the build_videos function:

78. user (2026-07-03T17:36:01.512Z)

```

80         for mwd in (1.0, 2.0, 3.0)
81         for mc in (0, 1, 2, 3)
82     ]
83
84
85     def build_videos(specs: List[Dict]) -> List[Dict]:
86         videos = []
87         for s in specs:
88             pf = make_local_predict_fn(
89                 s["trajectory"], float(s["fps"]), float(s["frame_width"])
90             )
91             gts = load_golden_gts(s["labels"])
92             if not gts:
93                 raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
94             videos.append(
95                 {"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts": gts}
96             )
97         return videos
98
99

```

79. assistant (2026-07-03T17:36:05.464Z)

Good! Line 91 calls `load_golden_gts(s["labels"])`, so `calibrate_local` requires labels. This is correct? it calls `load_resolved_manifest` with default arguments (which require both trajectory and labels).

Now let me make sure I've found all the bugs. Let me do a final check on the JSON exception handling:

Actually, I realize that catching `json.JSONDecodeError` alongside `ValueError` is not wrong? it's just redundant since `JSONDecodeError` is a `ValueError`. It's fine code style, if a bit verbose.

Let me look one more time at whether there are any issues with the test helpers in the test files:

80. user (2026-07-03T17:36:05.696Z)

```
<tool_use_error>InputValidationError: Read failed due to the following issue:  
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>
```

81. user (2026-07-03T17:36:08.047Z)

```
203  
204     out_dir = str(tmp_path / "out")  
205     os.makedirs(out_dir)  
206     for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):  
207         (tmp_path / name).write_text("x\n", encoding="utf-8")  
208     manifest = [  
209         {  
210             "name": "alpha",  
211             "trajectory": "a.csv",  
212             "fps": 30.0,  
213             "frame_width": 1920.0,  
214             "labels": "a.rallies.csv",  
215         },  
216         {  
217             "name": "beta",  
218             "trajectory": "b.csv",  
219             "fps": 30.0,
```

82. assistant (2026-07-03T17:36:12.549Z)

Good! The test helper writes the required trajectory and label files, so the manifest can resolve them. This looks correct.

Now let me finalize my analysis. I've found one clear bug:

****PRIMARY BUG**:** Line 231 in `\_candidate\_dirs` - the manifest_dir is included first in the candidate directories list, which could cause shadowing if a file with the same basename exists in both the manifest directory AND the standard Annotation Setup directories. The correct file in Annotation Setup could be shadowed by a file in manifest_dir.

Let me see if there are any other issues I should report:

Looking at the code once more, I think the main issue is the shadowing problem. Let me also check if there's any issue with how `--require` works in the CLI:

83. user (2026-07-03T17:36:12.963Z)

```
<tool_use_error>InputValidationError: Read failed due to the following issue:  
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>
```

84. user (2026-07-03T17:36:15.486Z)

```
304     p.add_argument(  
305         "--require",  
306         action="append",  
307         choices=sorted(_FIELD_DIR),  
308         default=None,  
309         help="path field to require; repeatable. Defaults to trajectory and labels.",  
310     )  
311     p.add_argument(  
312         "--min-count",  
313         type=int,
```

85. assistant (2026-07-03T17:36:21.039Z)

There's a subtle bug here! On line 306-308, the `--require` argument has `action="append"` and `default=None`. This means:

- If `--require` is not specified, `args.require` will be `None` (not `[]`)
- If `--require` is specified once, `args.require` will be a list with one element
- If `--require` is specified multiple times, `args.require` will be a list with multiple elements

But then on line 327, the code does:

```
```python
required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
```
```

This is correct?if `args.require` is `None`, it uses `DEFAULT_REQUIRED_FIELDS`. But there's a subtle issue: what if the user specifies `--require trajectory --require labels` explicitly? Then `args.require` will be `["trajectory", "labels"]`, which is fine. But if they specify only `--require trajectory`, then `args.require` will be `["trajectory"]`, and `DEFAULT_REQUIRED_FIELDS` is NOT used. This is actually correct behavior?the user explicitly requested only trajectory to be required.

So there's no bug here.

Now let me prepare my final report. I've identified 1 clear bug and found the code to be generally well-written otherwise.

```
```json
[
 {
 "file": "backend/eval/golden_manifest.py",
 "line": 231,
 "summary": "Manifest directory checked first in candidate search path, risking file shadowing",
 "failure_scenario": "If a file with the same basename exists in both manifest_dir (repo/output/) and the standard Annotation Setup directories, the manifest_dir version will be selected first, potentially loading the wrong file. For example, if both repo/output/trajectory.csv and ../Annotation Setup/Trajectories/trajectory.csv exist, the output/ version shadows the intended Annotation Setup version."
 }
]
```
```